BrandProof AI — Round 2 report

Tagline: Trusted Content. Stronger Brands.

Problem and target users

Organizations publish content through employees, agencies, freelancers, and AI tools. Manual review across documents, email, and shared drives makes incorrect facts, unsupported claims, inconsistent terminology, and stale approvals easy to miss. BrandProof targets content creators, brand reviewers, and brand administrators who need an evidence-backed review trail.

Proposed solution and actual workflow

The implemented MVP provides a seeded synthetic workspace for NovaTech X1. A creator opens an advertisement, runs an audit, sees claim-level structured findings, opens the approved evidence source, edits the draft, saves a new version, re-audits, submits it, and receives a human reviewer decision. The approved version has a SHA-256 content hash and passport. When a fact version changes, dependency records identify the affected content in Brand Drift.

Implemented features

See `features_completed.md` for the complete status list. The main completed path is dashboard → content audit → exact evidence → new version → human review → passport → drift. The mandatory 7000 mAh, waterproof, and superlative example is seeded and tested.

Screenshots

The product is implemented at `frontend/app/page.tsx` with a responsive dashboard, audit workspace, evidence cards, knowledge base, review queue, content history, and drift radar. Browser screenshot capture could not be completed in this Linux environment because the available browser-control display lacks Xvfb. The report therefore includes the source path and exact local preview instructions rather than fabricated screenshots. Run the app and open `http://localhost:3000` to capture screenshots for a final submission package.

Technology and architecture

The frontend is Next.js 16.4.3, TypeScript, React, and CSS. The backend is FastAPI, Pydantic, SQLAlchemy, pypdf, Alembic, Pillow, and pytesseract. SQLite is the zero-config default; PostgreSQL/pgvector is represented in Docker Compose for deployment-shaped environments. The audit engine uses exact structured retrieval for facts and policies, with explicit rules-only fallback and an optional Gemini adapter. Details are in `technology_and_implementation.md`.

AI processing and data flow

The current pipeline normalizes submitted text, extracts and persists claims with positions, retrieves relevant approved facts and policies, runs deterministic validation and administrator-authored compiled rules, emits structured findings, validates evidence ownership, persists dependencies, and returns the audit to the UI. When Gemini configuration is present, a real HTTP adapter sends retrieved context for semantic suggestions, validates structured JSON, and visibly falls back on provider errors. Without a key, the UI explicitly reports rules-only mode.

Testing results

Executed checks:

- Backend `DATABASE_URL=sqlite:///./test_brandproof.db .venv/bin/pytest -q`: **11 passed**.
- Backend `.venv/bin/python -m py_compile app/*.py tests/*.py`: **passed**.
- Frontend `npx tsc --noEmit`: **passed**.
- Frontend `npm run build`: **passed** after upgrading Next.js from 15.1.6 to 16.4.3 because the initial Next 15 binary produced a bus error in this environment.
- Live API smoke check: `/api/health`, `/api/dashboard`, and `/` returned HTTP 200 while the local services were running.
- Fresh Alembic migration check: **passed** with `alembic upgrade head` and `alembic check`.
- Docker build/run: **not run** because Docker is not installed in the WSL environment.
- Browser screenshot journey: **not run** because computer-control Xvfb is unavailable.

Security and responsible design

Secrets remain environment-based. Uploads restrict extensions and size, use safe basename handling, reject empty extraction, and do not render uploaded HTML. The app distinguishes approved from unreviewed source material, keeps human review separate from AI/rules analysis, exposes uncertainty, and uses fictional demo data. Production authentication and provider-retention guarantees are not implied.

Known limitations

Production authentication and tenant provisioning, visual asset comparison, semantic vector retrieval, object storage, and background jobs remain incomplete. OCR requires a separately installed Tesseract executable. The Gemini adapter exists but no live provider key was available for an external call test. SQLite is the tested local path. PostgreSQL/pgvector is deployment preparation rather than an executed environment check.

Round 3 improvements

The official roadmap contains exactly three substantial improvements: advanced evidence-backed claim verification; verifiable proof-carrying content; and proactive Brand Drift Radar. See `round3_improvements.md` for current behavior, gaps, planned architecture, and evaluation tests.

Verification snapshot

Check	Result
Backend automated tests	11 passed
Python compile check	Passed
Frontend TypeScript	Passed
Next.js production build	Passed
Live API smoke check	HTTP 200

Docker / browser capture	Environment unavailable